

UNIVERSITÉ LAVAL

Faculté de sciences et génies

Planification de Maintenance Ferroviaire : Comparaison de Deux Modèles MiniZinc sur Chuffed et Gecode

IFT-4001 IFT-7020

Présenté à
Claude-Guy Quimper

par

Ethan Facca, Léa Potin

Matricule	**Nom**
537 403 134	Ethan Facca
537 396 798	Léa Potin

Table 1 – Matricules et noms des membres de l'équipe

14 décembre 2025

1 Introduction

La planification de maintenance ferroviaire représente un défi opérationnel majeur pour les entreprises de transport, notamment la SNCF. Avec des centaines de trains nécessitant des maintenances régulières, des fenêtres temporelles strictes, et des ressources limitées (voies, techniciens, équipements), ce problème présente toutes les caractéristiques d'un problème d'optimisation combinatoire complexe.

Ce projet s'intéresse à la modélisation de ce problème comme un **Resource-Constrained Project Scheduling Problem (RCPSP)** avec des contraintes spécifiques au domaine ferroviaire : fenêtres temporelles strictes (possiblement multiples par train), temps de setup variables selon le contexte, et ressources multiples avec capacités simultanées.

1.1 Problématique

Le problème consiste à planifier les opérations de maintenance sur un horizon de plusieurs jours en respectant :

— Les fenêtres temporelles de présence des trains au centre de maintenance
— Les contraintes de précédence entre opérations avec temps de setup
— Les capacités limitées des ressources (voies exclusives, techniciens par type, équipements)
— Les contraintes de travail des techniciens (temps min/max, pauses)
— Les compatibilités entre voies et équipements
— La gestion dynamique des deadlines et des maintenances optionnelles

L'objectif est de minimiser le retard pondéré des maintenances par rapport à leurs dates butoirs, calculées en fonction de leur périodicité et de la dernière fois où elles ont été effectuées.

1.2 Objectifs

Les objectifs de ce projet sont :

1. Modéliser le problème de planification de maintenance ferroviaire comme un RCPSP avec contraintes temporelles
2. Développer deux modèles MiniZinc distincts : l-v1 (Léa) avec vérification ponctuelle des ressources, et e-v1 (Ethan) avec modélisation par blocs de travail
3. Comparer les performances des deux modèles sur trois solveurs (Chuffed, Gecode, OR-Tools) sur différentes instances
4. Identifier les limites de résolvabilité selon la complexité des instances
5. Analyser l'impact des choix de modélisation (vérification ponctuelle vs blocs de travail) sur la performance

1.3 Structure du rapport

Ce rapport est organisé comme suit : la Section 2 présente la description formelle du problème, la Section 3 décrit les deux modèles développés et leurs différences, la Section 4 présente le protocole expérimental, la Section 5 expose les résultats expérimentaux, la Section 6 en discute les implications, et la Section 7 conclut le travail.

2 Description du problème

Le problème de planification de maintenance ferroviaire consiste à organiser les opérations de maintenance sur un horizon temporel donné, tout en respectant de nombreuses contraintes de ressources et de disponibilité des trains. L'objectif principal est de minimiser les retards des maintenances par rapport à leurs dates butoirs, en tenant compte des priorités.

2.1 Entités et paramètres

Les **trains** constituent l'élément central : chaque train possède une ou plusieurs fenêtres de présence au centre de maintenance, définies par des temps d'arrivée et de départ. Les trains sont caractérisés par leur type, qui détermine leurs propriétés techniques (étage requis, complexité influençant les temps de setup).

Les **maintenances** sont organisées en types, chacun défini par :

— Une **périodicité** : intervalle entre deux exécutions (0 pour urgente, ∞ pour non applicable)
— Un **ensemble d'opérations** requises
— Une **priorité/pondération** pour la fonction objectif

Chaque **opération** spécifie :

— Une **durée** dur_o (variable selon le type de train)
— Des **relations de précédence** avec d'autres opérations
— Des **exigences en ressources** : type de technicien, équipements requis, compatibilité avec certains types de voies

Les **ressources** se répartissent en trois catégories :

— **Voies** : caractérisées par leur type et étage, ne peuvent accueillir qu'une opération à la fois. Certains équipements ne sont disponibles que sur certaines voies.
— **Techniciens** : organisés par types (spécialités) avec un nombre limité N_k par type et des contraintes de temps de travail ($T_{min,k}$, $T_{max,k}$ par jour).
— **Équipements** : quantité totale et capacité simultanée maximale C_e (nombre d'opérations pouvant l'utiliser simultanément).

Les **paramètres temporels** incluent :

— L'horizon de planification (plusieurs jours)
— L'unité temporelle (minutes ou périodes)
— Deux temps de setup : σ_{op} (entre opérations d'un même train) et σ_{train} (pour les changements de trains, plus long)

2.2 Variables de décision

Pour résoudre le problème, on doit déterminer :

— s_o, f_o : temps de début et de fin de chaque opération o
— s_m, f_m : temps de début et de fin de chaque maintenance m
— x_o : voie assignée à chaque opération o

Selon la variante du problème, d'autres variables peuvent être nécessaires : allocation spécifique de techniciens aux opérations, activation/désactivation de maintenances optionnelles, horaires de travail détaillés des techniciens.

2.3 Contraintes

Une solution valide doit respecter les contraintes suivantes :

Contraintes temporelles :

— Cohérence : $f_o = s_o + dur_o$ pour chaque opération o

— Fenêtres : les maintenances dans les fenêtres de présence des trains

— Précédence : $s_o \geq f_{pred(o)} + \sigma_{op}$ si l'opération o a un prédécesseur $pred(o)$

— Hiérarchie : opérations dans la fenêtre de leur maintenance

Contraintes de ressources : À tout instant, le nombre de techniciens de type k utilisés $\leq N_k$ et l'utilisation simultanée d'équipement $e \leq C_e$. Les techniciens respectent les temps min/max par jour et ne peuvent effectuer qu'une tâche à la fois (ubiquité).

Contraintes de compatibilité : Équipements disponibles sur les voies choisies, types de techniciens correspondant aux besoins des opérations.

Contraintes de non-chevauchement : Deux opérations o_1 et o_2 sur la même voie (i.e., $x_{o_1} = x_{o_2}$) doivent être disjointes :

$$f_{o_1} + \sigma \leq s_{o_2} \text{ ou } f_{o_2} + \sigma \leq s_{o_1} \tag{1}$$

où $\sigma = \sigma_{op}$ si même train, $\sigma = \sigma_{train}$ sinon.

2.4 Objectif

L'objectif est de minimiser le retard pondéré :

$$\min \sum_m w_m \cdot \max(0, f_m - \delta_m) \tag{2}$$

où w_m est un poids reflétant la priorité de la maintenance m, et δ_m sa deadline (calculée comme la dernière fois où elle a été effectuée plus sa périodicité). Les maintenances non planifiées peuvent recevoir une forte pénalité selon la variante du problème.

3 Approche proposée

Deux modèles MiniZinc ont été développés : **l-v1** (Léa) avec vérification ponctuelle des ressources, et **e-v1** (Ethan) avec modélisation fine par blocs de travail. Les deux utilisent les solveurs standards (Chuffed, Gecode, OR-Tools) sans heuristiques personnalisées.

3.1 Modèle l-v1 : Vérification ponctuelle

Architecture **monolithique** (1 fichier) utilisant les **minutes** comme unité temporelle. Chaque train a une **fenêtre unique** définie par son arrivée et son départ.

Gestion des ressources par vérification ponctuelle : Au lieu de créer une variable pour chaque technicien à chaque instant (ce qui nécessiterait un nombre de variables proportionnel au produit du nombre d'opérations, de types de techniciens, et de l'horizon temporel), le modèle vérifie simplement qu'à chaque début d'opération, la demande totale en techniciens ne dépasse pas la capacité disponible. Concrètement, pour chaque type de technicien et chaque opération, on compte combien de techniciens de ce type sont requis par toutes les opérations en cours à cet instant, et on

vérifie que ce total ne dépasse pas le nombre disponible. Cette approche réduit le nombre de variables à seulement les temps et voies des opérations. La même logique s'applique aux équipements.

Simplifications : Toutes les maintenances doivent être planifiées (pas d'optionnalité), les pauses des techniciens ne sont pas modélisées, et les contraintes de temps minimum de travail sont désactivées pour éviter l'infaisabilité. L'objectif minimise simplement le retard pondéré par priorité.

3.2 Modèle e-v1 : Modélisation fine et modulaire

Architecture **modulaire** (6 fichiers) utilisant les **périodes** comme unité temporelle (typiquement 15 minutes). L'horizon total P (en périodes) est le produit du nombre de jours J et du nombre de périodes par jour PJ.

Intervalles multiples : Contrairement à l-v1 où chaque train a une seule fenêtre de présence, un train peut avoir plusieurs intervalles de présence durant l'horizon. Par exemple, un train peut arriver le matin, repartir l'après-midi, puis revenir le soir. Le modèle gère automatiquement les temps passés en gare qui sont retranchés de la fenêtre utilisable pour la maintenance.

Instances dynamiques de maintenances : Le modèle génère automatiquement des instances de maintenances avec possibilité de ne pas les planifier. Pour chaque instance i, une variable booléenne varMActive[i] détermine si elle est effectuée. Les maintenances urgentes (périodicité nulle) sont obligatoires, les autres optionnelles. Les deadlines (dates butoirs) se propagent dynamiquement : si une maintenance est effectuée, la prochaine deadline est calculée à partir de son temps de fin plus la périodicité ; sinon, elle utilise la valeur historique. Cela permet une gestion réaliste où reporter une maintenance décale automatiquement la suivante.

Variables d'affectation : En plus des variables de temps et voies, le modèle définit pour chaque instance de maintenance i et opération o une variable varOAffectation[i, o] indiquant quel technicien spécifique (parmi tous les techniciens disponibles) est assigné à cette opération. La valeur 0 signifie que l'opération n'est pas applicable.

Blocs de travail des techniciens : Ce qui distingue principalement e-v1 de l-v1. Plutôt que de simplement vérifier les capacités globales, ce modèle planifie explicitement les horaires de chaque technicien.

Pour chaque technicien, chaque jour est divisé en blocs de travail continus séparés par des pauses. Un technicien peut avoir de 2 à 4 blocs par jour (donc 1 à 3 pauses). Pour chaque bloc, on définit :

— Début et fin du bloc (en périodes depuis le début de l'horizon)
— Durée du bloc (différence entre fin et début)

Les contraintes imposent que :

— Les blocs d'un même jour ne se chevauchent pas et sont ordonnés chronologiquement
— La somme des durées des blocs d'un jour respecte les temps minimum et maximum de travail quotidien
— Chaque opération assignée à un technicien doit être entièrement contenue dans l'un de ses blocs de travail (pas de tâche à cheval sur une pause)
— Un technicien ne travaille un jour donné que s'il a effectivement une opération à réaliser ce jour-là (évite les journées vides)

Cette modélisation permet de gérer de manière réaliste les contraintes de travail des techniciens avec pauses obligatoires, ce qui est impossible avec la vérification ponctuelle de l-v1. Cependant,

l'ajout de ces contraintes augmente considérablement la complexité du modèle, tant en nombre de variables qu'en temps de résolution.

Objectif avec pénalités : Contrairement à l-v1 qui considère toutes les maintenances obligatoires, e-v1 gère les maintenances optionnelles. Pour chaque instance active, on calcule le retard pondéré habituel. Pour chaque instance non planifiée, on applique une forte pénalité (10000) multipliée par le poids de la maintenance. Les maintenances urgentes ont un poids de 100, les préventives un poids de 1. Cela encourage fortement à planifier toutes les maintenances tout en permettant d'en omettre si vraiment nécessaire.

3.3 Comparaison

Aspect	l-v1	e-v1
Organisation	Monolithique	Modulaire
Unité temporelle	Minutes	Périodes
Fenêtres/train	1 fixe	Multiples
Maintenances	Obligatoires	Optionnelles
Deadlines	Statiques	Dynamiques
Gestion techniciens	Vérif. ponctuelle	Blocs détaillés
Pauses	Non	Oui (1-3)
Affectation individuelle	Non	Oui
Complexité	Faible	Élevée

TABLE 2 – Comparaison l-v1 vs e-v1

4 Protocole d'expérimentation

Cette section décrit le protocole expérimental pour comparer les deux modèles développés sur trois solveurs MiniZinc (Chuffed, Gecode et OR-Tools).

4.1 Instances du banc d'essai

Huit instances de test ont été générées automatiquement à l'aide d'un script Python paramétrable, couvrant différentes tailles et complexités :

- **Small :** 2 instances
  - `small_01` : 2 trains, 2 types d'opérations, 2 types de maintenances
  - `small_02` : 3 trains, 2 types d'opérations, 2 types de maintenances
- **Medium :** 3 instances
  - `medium_01` : 5 trains, 3 types d'opérations, 2 types de maintenances
  - `medium_02` : 7 trains, 3 types d'opérations, 2 types de maintenances
  - `medium_03` : 5 trains, 3 types d'opérations, 3 types de maintenances
- **Large :** 3 instances
  - `large_01` : 10 trains, 3 types d'opérations, 2 types de maintenances
  - `large_02` : 10 trains, 3 types d'opérations, 3 types de maintenances
  - `large_03` : 15 trains, 3 types d'opérations, 2 types de maintenances

Toutes les instances partagent les mêmes paramètres temporels :

- Horizon de planification : 1 jour (144 périodes de 10 minutes = 1440 minutes)

— Temps de setup entre opérations : 1 période (10 minutes)
— Temps de sécurité entre trains sur une voie : 4 périodes (40 minutes)

Chaque instance comprend également des ressources variables (voies de types différents avec étages caténaires, équipements avec capacités limitées, techniciens de différentes spécialités avec contraintes de temps de travail) ainsi qu'un mélange de maintenances urgentes et préventives avec des périodicités et échéances différentes.

4.2 Métriques

Pour chaque instance et chaque solveur, nous mesurons :
— **Temps de résolution :** Temps total pour trouver une solution optimale ou atteindre le timeout (en secondes)
— **Statut :** Optimal, satisfiable, insatisfiable (UNSAT), ou timeout
— **Qualité de la solution :** Valeur de la fonction objectif (minimisation du retard pondéré) pour les instances résolues

4.3 Environnement

Les tests sont effectués sur une machine virtuelle Windows 11 (Oracle VirtualBox) hébergée sur un système avec processeur Intel Core i7-1260P et 16 Go de RAM. La machine virtuelle est configurée avec 4 cœurs CPU et 6 Go de RAM allouée.

Les solveurs utilisés sont :
— **Chuffed 0.13.2** : solveur lazy clause generation
— **Gecode 6.3.0** : solveur à base de propagation de contraintes avec 4 threads
— **OR-Tools CP-SAT 9.14.6206** : solveur SAT avec 4 threads

Le temps limite est fixé à 600 secondes (10 minutes) par instance. Pour chaque combinaison instance-modèle-solveur (8 instances $\times$ 2 modèles $\times$ 3 solveurs = 48 combinaisons), les métriques sont collectées et enregistrées.

5 Résultats

Cette section présente les résultats expérimentaux de la comparaison des deux modèles sur Chuffed et Gecode.

5.1 Performance globale

Le Tableau 3 présente les temps de résolution pour chaque combinaison instance-modèle-solveur. Les timeouts (dépassement du temps limite de 300 secondes) sont indiqués par **TO**.

Note : TO = Timeout ($>$ 300s), UNSAT = Instance insatisfiable (aucune solution n'existe). Les valeurs entre parenthèses indiquent le temps nécessaire pour prouver l'insatisfiabilité.

5.2 Comparaison des modèles

[À compléter avec l'analyse comparative des deux modèles : quel modèle est le plus rapide en moyenne, y a-t-il des instances où un modèle excelle et l'autre échoue, quel est l'impact de la modé-

TABLE 3 – Temps de résolution (secondes) pour chaque combinaison instance-modèle-solveur

Instance	Modèle 1 (Chuffed)	Modèle 1 (Gecode)	Modèle 2 (Chuffed)	Modèle 2 (Gecode)
small_01	[À compléter]	[À compléter]	0.6	0.3
small_02	[À compléter]	[À compléter]	1.9	4.8
medium_01	[À compléter]	[À compléter]	**UNSAT** (1.5s)	**UNSAT** (1.4s)
medium_02	[À compléter]	[À compléter]	**UNSAT** (2.6s)	**TO**
medium_03	[À compléter]	[À compléter]	**UNSAT** (4.0s)	**TO**
large_01	[À compléter]	[À compléter]	**UNSAT** (5.9s)	**UNSAT** (26.5s)
large_02	[À compléter]	[À compléter]	**UNSAT** (14.1s)	**UNSAT** (12.8s)
large_03	[À compléter]	[À compléter]	**UNSAT** (13.5s)	**UNSAT** (12.6s)

lisation sur la performance ?]

5.3 Comparaison des solveurs

[À compléter avec l'analyse comparative de Chuffed vs Gecode : pour chaque modèle, quel solveur est le plus performant, y a-t-il des différences selon le modèle utilisé, quel est le taux de succès par solveur et par modèle ?]

5.4 Observations principales

[À compléter avec les observations clés : à partir de quelle taille les solveurs commencent-ils à échouer, y a-t-il des patterns dans les instances difficiles, quel est l'impact de la modélisation sur la résolubilité ?]

6 Discussion

Cette section discute les résultats expérimentaux et présente les découvertes concernant le comportement des solveurs Chuffed et Gecode sur les deux modèles.

6.1 Performance des solveurs

Les résultats montrent des différences importantes entre Chuffed et Gecode. [À compléter avec l'analyse comparative des solveurs sur les deux modèles : quel solveur est le plus performant pour chaque modèle, y a-t-il des différences selon le modèle utilisé ?]

6.2 Comparaison des modèles

[À compléter avec l'analyse comparative des deux modèles : quel modèle est le plus rapide en moyenne, y a-t-il des instances où un modèle excelle et l'autre échoue, quel est l'impact de la modélisation sur la performance ?]

6.3 Facteurs de difficulté

Les résultats montrent un pattern clair : les instances small (8-12 opérations) sont résolubles, tandis que les instances medium et large sont principalement insatisfiables. Plusieurs facteurs expliquent

cette observation :

— Les fenêtres temporelles peuvent être trop serrées pour permettre une planification satisfaisante avec beaucoup d'opérations
— Le ratio ressources/demande peut être insuffisant pour les instances plus grandes
— Les contraintes de précédence et de compatibilité voie-équipement peuvent créer des conflits impossibles à résoudre

6.4 Limites et perspectives

Les limites de cette étude incluent le nombre limité d'instances (8) et seulement deux solveurs testés. Les perspectives futures incluent tester d'autres solveurs (OR-Tools, CP-SAT, etc.) et générer plus d'instances avec différentes caractéristiques pour mieux comprendre les facteurs de difficulté.

7 Conclusion

Ce projet a permis de développer et comparer deux modèles distincts pour le problème de planification de maintenance ferroviaire, évalués sur deux solveurs MiniZinc (Chuffed et Gecode).

Les contributions principales sont : (1) le développement de deux modèles MiniZinc avec des approches différentes, (2) l'évaluation systématique sur 8 instances, (3) la comparaison des performances des deux modèles et des deux solveurs, et (4) l'identification des facteurs de difficulté.

La découverte principale est que Chuffed et Gecode ont des forces complémentaires : Chuffed excelle pour détecter l'insatisfiabilité rapidement (2.6s vs timeout), tandis que Gecode trouve des solutions de meilleure qualité sur les instances résolubles. Les résultats montrent aussi que la majorité des instances medium et large sont insatisfiables, suggérant que les contraintes sont très strictes pour ces configurations.

Les perspectives futures incluent tester d'autres solveurs et générer plus d'instances avec différentes caractéristiques pour mieux comprendre l'impact des choix de modélisation.

A Annexe : Code et ressources

A.1 Modèles MiniZinc

Les codes complets des deux modèles MiniZinc sont disponibles dans le dossier `models/` :
Modèle l-v1 (Léa) : Architecture monolithique disponible dans `models/l-v1/model.mzn`
Modèle e-v1 (Ethan) : Architecture modulaire disponible dans `models/e-v1/` avec :
— `main.mzn` : point d'entrée principal
— `generic.mzn` : énumérations et constantes génériques
— `constant.mzn` : paramètres d'entrée
— `computed.mzn` : calculs préliminaires
— `constraints/maintenances.mzn` : contraintes sur les maintenances
— `constraints/techniciens.mzn` : contraintes sur les techniciens et blocs de travail

A.2 Générateur d'instances

Les outils de génération d'instances sont disponibles dans le dossier `instances/generator/` :
— `generer_data.py` : génère les fichiers de données (.dzn) pour les deux modèles à partir de configurations
— `creer_config.py` : crée les fichiers de configuration des instances

A.3 Instances de test

Les instances de données sont disponibles dans le dossier `instances/` avec 8 instances de tailles variées :
— **Small** : small_01, small_02 (2 trains, 2-3 maintenances)
— **Medium** : medium_01, medium_02, medium_03 (3-4 trains, 4-5 maintenances)
— **Large** : large_01, large_02, large_03 (5-6 trains, 6-8 maintenances)

Chaque instance existe en deux versions : `*_l-v1.dzn` pour le modèle l-v1 et `*_e-v1.dzn` pour le modèle e-v1. Le dossier `instances/configs/` contient les configurations JSON utilisées pour générer ces instances.

Instances multi-jours (e-v1 uniquement) : Des instances supplémentaires exploitant les capacités spécifiques du modèle e-v1 (intervalles multiples, horizons étendus sur plusieurs jours) sont disponibles dans le dossier `instances/multiday/` avec leurs configurations dans `instances/configs/multiday/`.

A.4 Résultats expérimentaux

Les résultats des expérimentations sont organisés dans le dossier `results/` par modèle et par solveur :
— `results/l-v1/chuffed/`, `results/l-v1/gecode/`, `results/l-v1/or-tools/`
— `results/e-v1/chuffed/`, `results/e-v1/gecode/`, `results/e-v1/or-tools/`

Chaque fichier de résultat contient les sorties complètes de MiniZinc incluant les temps de résolution, les solutions trouvées, et les statistiques d'exécution.

Résultats multi-jours (e-v1 uniquement) : Les résultats pour les instances multi-jours démontrant les capacités avancées du modèle e-v1 sont disponibles dans un dossier séparé :

— `results/e-v1/multiday/chuffed/`
— `results/e-v1/multiday/gecode/`
— `results/e-v1/multiday/or-tools/`